Contents

1 Support Vector Machines (SVM) **2**

1.1 Hyperplanes and Decision Boundary 2

1.2 A simple example 4

1.3 Hard-Margin Primal Problem 6

1.4 Hard-Margin Optimization Dual Problem (Lagrange Multipliers) 8

1.5 Soft-Margin Primal and Dual Problem 12

1.6 Linearly Separable and Non-linearly Separable 16

2 The Kernel Trick **20**

Supplementary Notes

These notes are intended to supplement (and hence the lack of diagrams) the following lectures on Support Vector Machines:

- **MIT 6.034 Artificial Intelligence, Fall 2010** – Patrick Winston
  `https://www.youtube.com/watch?v=_PwhiWxHK8o`
- **The Kernel Trick** – DataMListic
  `https://www.youtube.com/watch?v=N_RQj4OL1mg`

Chapter 1

Support Vector Machines (SVM)

1.1 Hyperplanes and Decision Boundary

The basic objective and principle of SVM is to formally define a hyperplane (called a line in two dimensions) such that it acts as a decision boundary that best separate the classes. The data points closest to the hyperplane are known as **support vectors**. The most optimal hyperplane will have the maximum distance from each of the support vectors on both sides of the decision boundary. The distance between the hyperplane and the support vectors is known as the **margin**. The goal is to maximize the width of the separation between support vectors of classes.

Suppose in Euclidean space $\mathbb{R}^2$, we have some positive or negative examples on the right and left side of the hyperplane respectively. We now define $\vec{w}$ as the vector perpendicular to the hyperplane. Suppose we have some unknown vector $\vec{u}$. We are interested if this vector $\vec{u}$ is in the right or left side of the hyperplane. Thus, we project the $\vec{u}$ onto $\vec{w}$, the higher this value, the more likely it will be on the right side of the hyperplane.

$$\vec{w} \cdot \vec{u} \geq c, \text{where } c \text{ is a constant.} \tag{1.1}$$

Or without loss of generality, we define the decision rule as:

$$\vec{w} \cdot \vec{u} + b \geq 0, \text{where } c = -b \tag{1.2}$$

Notice: $\vec{w}$ sets the direction/tilt, $\vec{b}$ sets the offset

We now require some constraints to determine $\vec{w}$ and b. We impose the following two constraints:

$$\begin{aligned} \vec{w} \cdot \vec{x_+} + b \geq 1 \\ \vec{w} \cdot \vec{x_-} + b \leq 1 \end{aligned} \tag{1.3}$$

where x_+ and x_- are any points on the positive and negative sample respectively

For mathematical convenience, we now introduce a variable y_i such that:

$$y_i = \begin{cases} +1, & \text{if } \vec{w} \cdot \vec{x_i} + b \geq 1 \\ -1, & \text{if } \vec{w} \cdot \vec{x_i} + b \leq 1 \end{cases}$$

Multiplying y_i into eq. (1.3), we obtain:

$$\begin{aligned} y_i(\vec{w} \cdot \vec{x_i} + b) \geq 1 \\ y_i(\vec{w} \cdot \vec{x_i} + b) \geq 1 \end{aligned}$$

Which can be represented by what we define as the **Hard-Margin SVM Constraint**:

$$\boxed{g_i(w,b) = y_i(\vec{w} \cdot \vec{x_i} + b) - 1 \geq 0} \tag{1.4}$$

For the positive margin and negative hyperplanes respectively where the support vector lies, we can represent those hyperplanes respectively as:

$$\vec{w} \cdot \vec{x_+} + b = 1$$
$$\vec{w} \cdot \vec{x_-} + b = -1$$

Therefore,

$$\vec{w} \cdot \vec{x_+} + b = 1$$
$$\vec{w} \cdot \vec{x_-} + b = -1$$
$$y_i(\vec{w} \cdot \vec{x_+} + b) = 1$$
$$y_i(\vec{w} \cdot \vec{x_-} + b) = 1$$

Therefore,

$$\boxed{y_i(\vec{w} \cdot \vec{x_i} + b) - 1 = 0} \tag{1.5}$$

Before proceeding, we will attempt some simple examples to obtain an intuitive understanding of eq. (1.3).

1.2 A simple example

Consider the two-dimensional Euclidean space $\mathbb{R}^2$ and we have the line:

$$y = mx + c$$
$$mx - y + c = 0$$

To convert this to hyperplane form, we note that the SVM hyperplane form can be defined as (noting: $\vec{x} = (x, y)$):

$$\vec{w} \cdot \vec{x} + b = 0 \tag{1.6}$$

Hence, $\vec{w} = (m, -1)$, $b = c$, note this does not take the form of eq. (1.2) since $b = c$ and not $b = -c$. We will realise it is much more convenient to adopt the the form given by eq. (1.2) because we want the "positive side" to correspond to the right side of the hyperplane.

Say now we have the line:

$$y = -5x + 2 \text{ or in hyperplane form: } \vec{w} \cdot \vec{x} + b = 0,$$
$$5x + y - 2 = 0$$
$$\vec{w} = (5, 1), b = -2$$
$$\vec{w} \cdot \vec{x} \geq 2, b = -c, \text{note this takes the form of eq. (1.2)}$$

Now we want to test if eq. (1.3) is true for two arbitary points $(2, 0)$ and $(-4, 0)$.

$$\vec{w} \cdot \vec{x} + b = 5(2) + 1(0) - 2 = 8 \text{ (positive side)}$$
$$\vec{w} \cdot \vec{x} + b = (-4) + 1(0) - 2 = -22 \text{ (negative side)}$$

But what if we had done this question differently in this way:

$$-5x + 2 - y = 0$$
$$\vec{w} = (-5, -1), b = 2$$
$$\vec{w} \cdot \vec{x} + b = -5(2) - 1(0) + 2 = -8 \text{ (negative side)}$$
$$\vec{w} \cdot \vec{x} + b = -5(-4) - 1(0) + 2 = 22 \text{ (positive side)}$$

Why has the result become inverted? Because we have to understand "positive side" corresponds to the direction the vector $\vec{w}$ points for $\vec{w} \cdot \vec{x} + b > 0$. In the first case, $\vec{w} = (5, 1)$ so positive corresponds to the upper right region. In the second case $\vec{w} = (-5, -11)$ so positive corresponds to the lower left region.

1.3 Hard-Margin Primal Problem

We now attempt to find the width between the two margin lines. Suppose we have two support vectors, $\vec{x_+}$ lying on the positive margin line and $\vec{x_-}$ lying on the negative margin line. $\vec{x_+} - \vec{x_-}$ will now be the vector starting and pointing from the negative margin line to the positive margin line and ending on it. However this vector is not perpendicular to the hyperplane, to ensure it is, we perform the following:

$$\text{Width of the margin } = (\vec{x_+} - \vec{x_-}) \cdot \frac{\vec{w}}{\|\vec{w}\|}$$

Note: $\|\vec{w}\|^2 = \vec{w} \cdot \vec{w}$

From eq. (1.5):

$$\vec{x_+} \cdot \vec{w} = 1 - b$$
$$-\vec{x_-} \cdot \vec{w} = 1 + b$$

$$(\vec{x_+} - \vec{x_-}) \cdot \frac{\vec{w}}{\|\vec{w}\|} = \frac{2}{\|\vec{w}\|}$$

$$\boxed{\text{Width of the margin } = \frac{2}{\|\vec{w}\|}}$$

We want to maximize this quantity $\frac{2}{\|\vec{w}\|}$, which is equivalent to maximizing $\frac{1}{\|\vec{w}\|}$, which is equivalent to minimizing $\|\vec{w}\|$. For mathematical convenience, this is again equivalent to minimizing $\frac{1}{2}\|\vec{w}\|^2$. Therefore we now define the **Hard-Margin Primal Problem**: (note: by Primal we mean the original optimization problem)

$$\boxed{\begin{array}{c} \min\limits_{\vec{w},b} \dfrac{1}{2}\|\vec{w}\|^2 \\ \text{s.t. } y_i(\vec{w} \cdot x_i + b) \geq 1 \end{array}} \tag{1.7}$$

The **Objective Function** is $\frac{1}{2}\|\vec{w}\|^2$, and it is what we are trying to minimise. The primal variables as shown are $\vec{w}$ and b.

Now we define:

$$p^* = \text{Optimal Primal Value} \tag{1.8}$$

And this is the smallest possible value of $\frac{1}{2}\|\vec{w}\|^2$ that satisfies the constraints, meaning its the solution value to the Primal Problem.

1.4 Hard-Margin Optimization Dual Problem (Lagrange Multipliers)

The situation we have at end is such that we want to now find the extremum of a function: eq. (1.7) with a constraint namely: eq. (1.5). This motivates us to use the Lagrange multipliers. Recall for Lagrange multipliers, the formula is given as (say we have a function $f(x)$):

$$\nabla f = \lambda \nabla g$$
$$\nabla f - \lambda \nabla g = 0$$

We now define the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x) \tag{1.9}$$

And we have to solve for:

$$\nabla_{x,\lambda} \mathcal{L} = 0$$

In our case, the Lagrangian is:

$$\mathcal{L}(w, b, \alpha) = f(w) - \sum_{i=1}^{l} \alpha_i g_i(w, b)$$

$$\mathcal{L} = \frac{1}{2}\|\vec{w}\|^2 - \sum_{i=1}^{l} \alpha_i \left[y_i(\vec{w} \cdot \vec{x_i} + b) - 1\right] \tag{1.10}$$

Where $f(w) = \frac{1}{2}\|\vec{w}\|^2$, $g_i(w, b) = y_i(\vec{w} \cdot \vec{x_i} + b) - 1$, l is the number of training samples

We now impose an additional constraint namely:

$$\alpha_i \geq 0, i = 1, ..., l \tag{1.11}$$

To understand why, suppose this constraint is satisfied, and $g_i(w, b) > 0$, then $-\alpha_i g_i(w, b) < 0$. However if this constraint is violated, then $g_i(w, b) > 0$, then $-\alpha_i g_i(w, b) > 0$. $\mathcal{L}$ can become huge if α grows too large.

We now define the **dual function**:

$$g(\alpha) = \inf_{w,b} \mathcal{L}(w, b, \alpha) \tag{1.12}$$

This function means for a fixed α, minimise L over w and b because recall that was our intention in (1.7).

Because of eq. (1.4) and eq. (1.11), $\sum_i^l \alpha_i g_i(w, b) \geq 0$, so:

$$\mathcal{L} = f(w) - \sum_i^l \alpha_i g_i(w, b) \leq f(w) \tag{1.13}$$

Now notice, because p^* is the smallest feasible value,

$$f(w) \geq p^*$$

And eq. (1.20), the smallest value of $\mathcal{L}$ must satisfy:

$$g(\alpha) = \inf_{w,b} \mathcal{L}(w, b, \alpha) \leq p^*$$

$$g(\alpha) \leq p^* \text{ for every } \alpha \geq 0 \tag{1.14}$$

The dual function gives a lower bound on the original minimization

problem. We however want to obtain the tightest lower bound because it is closest to p^*. We define this tightest lower bound as the **Dual Optimal Value**:

$$d^* = \max_{\alpha \geq 0} g(\alpha) \tag{1.15}$$

We now return to the Lagrangian in eq. (1.10) , take the derivative with respect to the primal variables and solve for:

$$\nabla_{\vec{w},b} \mathcal{L} = 0$$

$$\frac{\partial \mathcal{L}}{\partial \vec{w}} = \vec{w} - \sum_{i=1}^{l} \alpha_i y_i \vec{x_i} = 0$$

$$\boxed{\vec{w} = \sum_{i=1}^{l} \alpha_i y_i \vec{x_i}} \tag{1.16}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum \alpha_i y_i = 0$$

$$\boxed{\sum_{i=1}^{l} \alpha_i y_i = 0} \tag{1.17}$$

Substituting eq. (1.16) and eq. (1.17) into eq. (1.10), we obtain:

$$\begin{aligned}
\mathcal{L} &= \frac{1}{2}\left(\sum_{i=1}^{l} \alpha_i y_i \vec{x_i}\right) \cdot \left(\sum_{i=1}^{l} \alpha_j y_j \vec{x_j}\right) - \sum_{i=1}^{l} \alpha_i y_i \vec{x_i} \cdot \left(\sum_{i=1}^{l} \alpha_j y_j \vec{x_j}\right) - \sum_{i=1}^{l} \alpha_i y_i b + \sum_{i=1}^{l} \alpha_i \\
&= \frac{1}{2}\left(\sum_{i=1}^{l} \alpha_i y_i \vec{x_i}\right) \cdot \left(\sum_{i=1}^{l} \alpha_j y_j \vec{x_j}\right) - \sum_{i=1}^{l} \alpha_i y_i \vec{x_i} \cdot \left(\sum_{i=1}^{l} \alpha_j y_j \vec{x_j}\right) - b \sum_{i=1}^{l} \alpha_i y_i + \sum_{i=1}^{l} \alpha_i
\end{aligned}$$

$$\boxed{\mathcal{L} = \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j \vec{x_i} \cdot \vec{x_j}} \tag{1.18}$$

We realise the optimization depends only on the dot product of pairs of samples.

Substituting eq. (1.16) into eq. (1.2), we obtain:

$$\sum_{i=1}^{l} \alpha_i y_i \vec{x_i} \cdot \vec{u} + b \geq 0, \text{ for positive sample}$$

Now, the decision depends similarly on the dot product of the sample vectors and the unknown vector $\vec{u}$.

Recall also, eq. (1.18) is equivalent to $g(\alpha)$ and we want to maximise it to obtain the dual optimal value shown in eq. (1.15). Therefore, we now define the **Hard-Margin Dual Problem** as (called Dual because of the optimization problem $\max_{\alpha \geq 0} g(\alpha)$ from eq. (1.15) is constructed from the Primal Problem in eq. (1.7):

$$\boxed{\begin{gathered} \max_{\alpha} \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j \vec{x_i} \cdot \vec{x_j} \\ \text{s.t. } \alpha_i \geq 0, i = 1, ..., l \\ \sum_{i=1}^{l} \alpha_i y_i = 0 \end{gathered}} \tag{1.19}$$

Recall we mentioned the Optimal Primal Value and Optimal Dual Value in eq. (1.8) and eq. (1.15) respectively. Now, because each $g(\alpha)$ is a lower bound as mentioned in eq. (1.14), even the best lower bound is still a lower bound and hence we define the the **Duality Gap (Weak Duality in this**

case) as:

$$d^* \leq p^* \tag{1.20}$$

It so happens that in the case for SVM, d^* is equivalent to p^* (for reasons we will not go into) and thus we define **Strong Duality** as:

$$d^* = p^* \tag{1.21}$$

This is favourable for us because that means solving the Dual Problem gives us the exact same optimal value for solving the Primal Problem and we can solve whichever is computationally easier. In the next chapter we will apply the Kernel trick, considering the Dual depends only on the dot product of the pair of samples, and solving this Kernelized Dual will still give the correct Primal Optimum.

1.5 Soft-Margin Primal and Dual Problem

Now, consider real world data often containing noise and outliers, thereby disobeying the Hard-Margin SVM constraint namely, eq. (1.4). To resolve this, we add slack variables ζ_i to the constraint, which now becomes the Soft-Margin SVM Constraint:

$$\boxed{g_i(w, b, \zeta_i) = y_i(\vec{w} \cdot \vec{x_i} + b) - 1 + \zeta_i \geq 0} \tag{1.22}$$

If $\zeta_i = 0$, the point satisfies the margin rule. If $0 < \zeta_i < 1$, the point is correctly classified but inside the margin. If $\zeta_i \geq 1$, the point is misclassified. So assume a point violates the margin badly, we can set $\zeta_i = 1000$ and eq. (1.22) can easily be satisfied, say by setting $\vec{w} = 0$, which makes $\|w\|^2 = 0$, minimising the objective function perfectly. But this

creates a problem, namely, the decision boundary (eq. (1.3)) becomes meaningless because it no longer depends on $\vec{x}$. The width of the margin also becomes infinite, which is not useful. In other words, the classifier collapses and no longer works. A solution is to allow these violations of noisy data, but add or impose a "cost" on it. We use a technique called L1 regularization so that large values of ζ_i would "cost" something. Therefore we add the following term to the objective function:

$$C\sum_{i=1}^{l}\zeta_i$$

And we obtain a new objective function and define the **Soft-Margin Primal Problem** (The primal variables as shown below are w, b and ζ_i):

$$\boxed{\begin{array}{c}\min\limits_{w,b,\zeta_i}\frac{1}{2}\|\vec{w}\|^2+C\sum\limits_{i=1}^{l}\zeta_i\\ \text{s.t. } y_i(\vec{w}\cdot x_i+b)\geq 1-\zeta_i,\zeta_i\geq 0, i=1,...,l\end{array}} \tag{1.23}$$

Now observe the objective function, there is a competition between making $\|\vec{w}\|$ small (wide margin) and making ζ_i small (fewer violations). If $\|\vec{w}\|$ is small, a wider margin results in more points falling inside the margin and hence higher ζ_i. If $\|\vec{w}\|$ is big, a narrower margin results in less points falling inside the margin and hence lower ζ_i.C determines the relative importance, meaning how much do we care about slack compared to margin size, it determines the how strong the slack is penalized. To illustrate, suppose that slack becomes expensive, meaning C is extremely large, which increases the objective function. To avoid that, $\|\vec{w}\|$ can be increased, the margin becomes smaller, so more points can be correctly classified. In other words, the model will try to classify that outlier correctly and therefore overfitting occurs. On the other hand, suppose slack is cheap, meaning C is extremely small, the optimizer thinks that it's not worth to increase $\|\vec{w}\|$ to fix the violations of the outliers so it prefers to keep $\|\vec{w}\|$ small, the margin

becomes wider, so now there are more points inside the margin and hence misclassified. In other words, the model does not attempt to classify that outlier correctly and therefore underfitting occurs.

Now similarly, we apply the Lagrange multipliers method to the Soft-Margin Primal Problem. We first write the Lagrangian, this time introducing an additional multiplier $\mu_i \geq 0$ because now there is an additional constraint to the slack variables (now there are 2 constraints). The Lagrangian can be written as:

$$\mathcal{L} = \frac{1}{2}\|\vec{w}\|^2 + C\sum_{i=1}^{l}\zeta_i - \sum_{i=1}^{l}\alpha_i\left[y_i(\vec{w}\cdot\vec{x_i}+b) - 1 + \zeta_i\right] - \sum_{i=1}^{l}\mu_i\zeta_i \tag{1.24}$$

All the steps here for soft-margin is the same as hard-margin but we shall skips some of the steps here, such as defining the primal optimum, dual function, dual optimum, using weak duality and strong duality. Instead, we proceed straight to taking the derivatives with respect to the primal variables. When taking the derivatives with respect to $\vec{w}$ and b, we obtain the exact same results as eq. (1.16) and eq. (1.17).

$$\boxed{\vec{w} = \sum_{i=1}^{l}\alpha_i y_i \vec{x_i}} \tag{1.25}$$

$$\boxed{\sum_{i=1}^{l}\alpha_i y_i = 0} \tag{1.26}$$

We now take the derivative with respect to one specific ζ_k where $i \neq k$, note the sum vanishes and only the $i = k$ term survives:

$$\frac{\partial\mathcal{L}}{\partial\zeta_k} = C - \alpha_k - \mu_k = 0$$

Or we can just write:

$$\frac{\partial \mathcal{L}}{\partial \zeta_i} = C - \alpha_i - \mu_i = 0$$

$$\alpha_i + \mu_i = C$$

And since we know $\alpha_i \geq 0$ and $\mu_i \geq 0$, therefore the following condition must hold true:

$$0 \leq \alpha_i \leq C$$

We now substitute eq. (1.25) and eq. (1.26) back into the Lagrangian namely: eq. (1.24). We first rewrite the Lagrangian as:

$$\mathcal{L} = \frac{1}{2}\|\vec{w}\|^2 - \sum_{i=1}^{l} \alpha_i \left[y_i(\vec{w} \cdot \vec{x_i} + b) - 1\right] + \sum_{i=1}^{l} \zeta_i (C - \alpha_i - \mu_i)$$

Since $C - \alpha_i - \mu_i = 0$

$$\mathcal{L} = \frac{1}{2}\|\vec{w}\|^2 - \sum_{i=1}^{l} \alpha_i \left[y_i(\vec{w} \cdot \vec{x_i} + b) - 1\right]$$

The final optimization problem we define as the **Soft-Margin Dual Problem** is almost completely similar to the soft-margin case, with the only change being the constraint changing from $\alpha_i \geq 0$ to $0 \leq \alpha_i \leq C$:

$$\boxed{\begin{gathered} \max_{\alpha} \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j \vec{x_i} \cdot \vec{x_j} \\ \text{s.t. } 0 \leq \alpha_i \leq C, i = 1, ..., l \\ \sum_{i=1}^{l} \alpha_i y_i = 0 \end{gathered}} \tag{1.27}$$

1.6 Linearly Separable and Non-linearly Separable

Everything we have done so far can be applied to linearly separable data. Take a look at Fig. 1.1, in this instance, all the data can be separated linearly by a straight line, in this case there is a optimal separating hyperplane. But what if this is not the case? To understand observe Fig. 1.2, there is clearly no way to linearly separate the positive and negative samples. To resolve this, we introduce a non-linear feature map $\phi(x) = (x, x^2)$, which embeds the data into $\mathbb{R}^2$, where linear separation becomes possible as we observe in Fig. 1.3 by the line $y = 12.5$. A more powerful application can be observed in Fig. 1.4, where there are samples on two concentric circles namely: $x^2 + y^2 = 9$ and $x^2 + y^2 = 25$. Similarly, we introduce a non linear feature map $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$, embedding the data into $\mathbb{R}^3$. The classes now become linearly separable by the hyperplane (a plane in this case) of the form $z = c$ where $9 < c < 25$ as illustrated by Fig. 1.5.

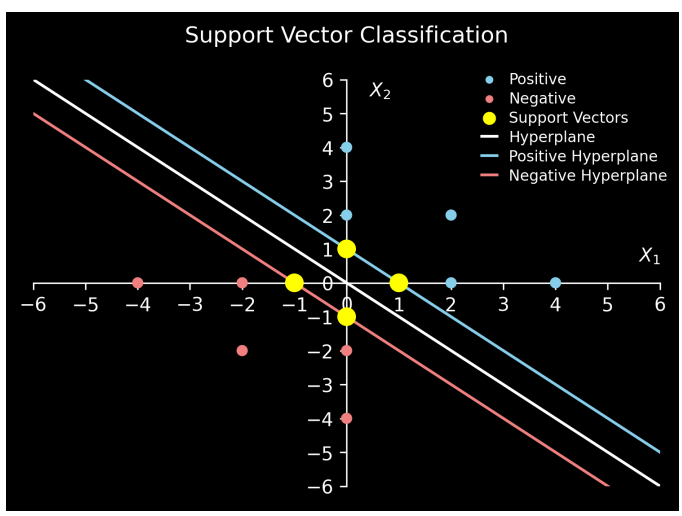

Figure 1.1: Support Vector Classification (Linearly Separable)

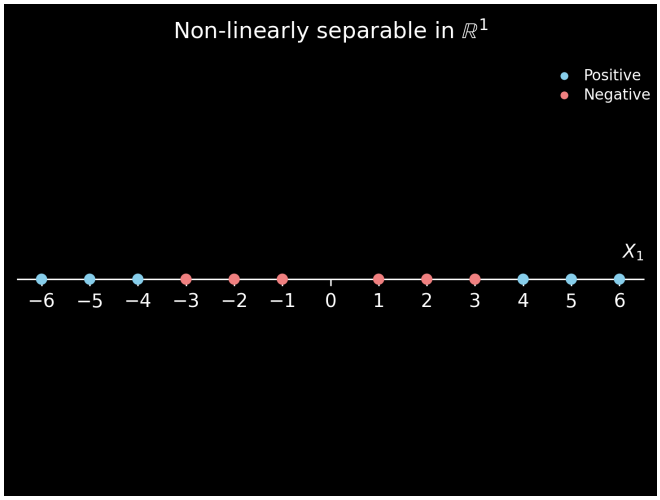

Figure 1.2: Support Vector Classification (Non-linearly separable in $\mathbb{R}^1$)

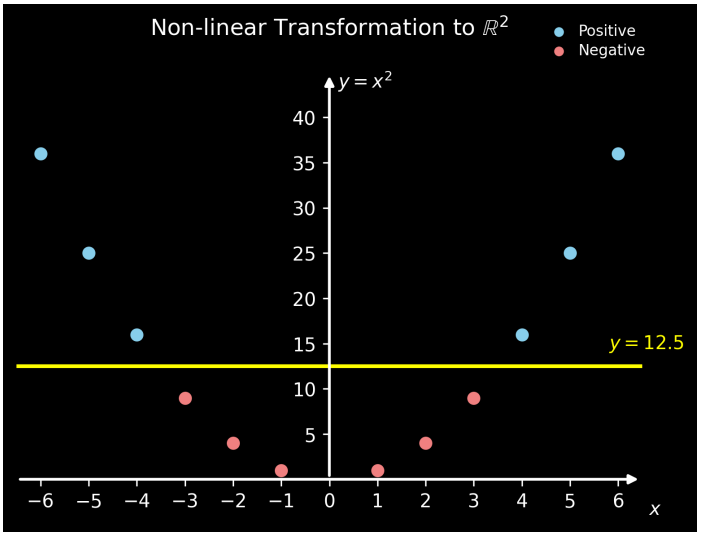

Figure 1.3: Support Vector Classification (Non-linear Transformation to $\mathbb{R}^2$)

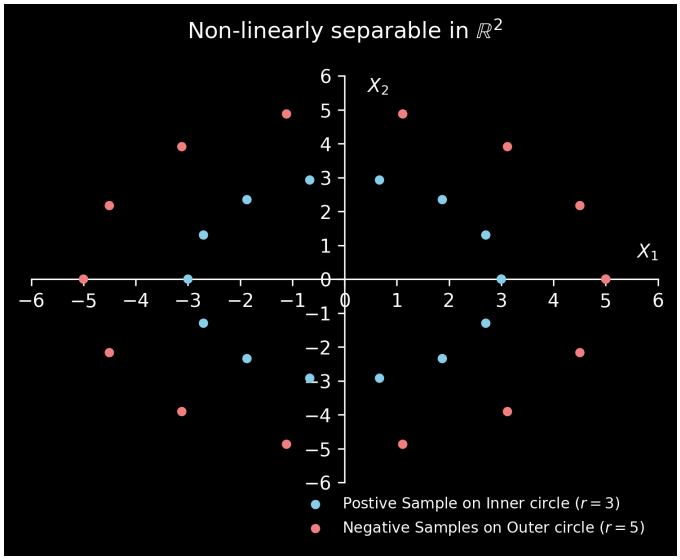

Figure 1.4: Support Vector Classification (Non-linear separable in $\mathbb{R}^2$)

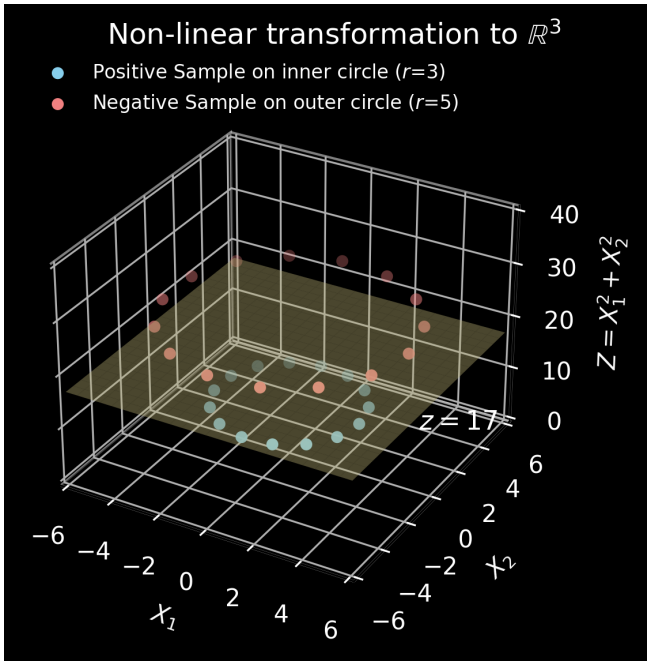

Figure 1.5: Support Vector Classification (Support Vector Classification (Non-linear Transformation to $\mathbb{R}^3$)

Chapter 2

The Kernel Trick

As mentioned in Section 1.6, we can transform our data into higher dimensions to ensure linear separability. However, what if the transformed space is very high-dimensional? This makes computation to be very expensive. To illustrate this suppose we have an original vector namely:

$$\vec{x} = (x_1, x_2)$$

Suppse we want all polynomial terms up to degree 3:

$$\text{Up to Degree 0: } (1)$$
$$\text{Up to Degree 1: } (x_1, x_2)$$
$$\text{Up to Degree 2: } (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$$
$$\text{Up to Degree 3: } (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3)$$

Therefore, the explicit polynomial feature map is a 10-dimensional vector, where computing in this high-dimensional space is expensive:

$$\phi(x_1, x_2) = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1 x_2 \\ x_2^2 \\ x_1^3 \\ x_1^2 x_2 \\ x_1 x_2^2 \\ x_2^3 \end{pmatrix}$$

In order to alleviate this problem, we now introduce the **Kernel Trick**, which is a method that allows us to avoid the need for the transformed data points, and instead compute the dot product in a high-dimensional feature space. Suppose we define a feature map:

$$\phi : \mathbb{R}^d \to \mathbb{R}^D \text{ ,where D may be very large}$$

Now we transform our data points and eq. (1.27) becomes (keeping in mind only the dot product is important here):

$$\max_{\alpha} \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j \phi(\vec{x_i}) \cdot \phi(\vec{x_j})$$

Instead of computing $\phi(\vec{x_i}) \cdot \phi(\vec{x_j})$, we define a **Kernel Function**:

$$K(\vec{x_i}, \vec{x_j}) = \phi(\vec{x_i}) \cdot \phi(\vec{x_j}) \tag{2.1}$$

Therefore, our optimization problem becomes the **Kernelized Dual Problem**:

$$\max_{\alpha} \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j K(\vec{x_i} \cdot \vec{x_j})$$

To illustrate, consider a Polynomial Kernel of degree 2, suppose we have the vectors:

$$\vec{x} = (x_1, x_2), \vec{y} = (y_1, y_2)$$

And we have the transformed feature vector:

$$\phi(x_1, x_2) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2) \tag{2.2}$$

$$\phi(y_1, y_2) = (1, \sqrt{2}y_1, \sqrt{2}y_2, y_1^2, \sqrt{2}y_1y_2, y_2^2) \tag{2.3}$$

Now compute the dot product:

$$\begin{aligned}
\phi(\vec{x}) \cdot \phi(\vec{y}) &= 1 \cdot 1 + \sqrt{2}x_1 \cdot \sqrt{2}y_1 + \sqrt{2}x_2 \cdot \sqrt{2}y_2 + x_1^2 \cdot y_1^2 + \sqrt{2}x_1x_2 \cdot \sqrt{2}y_1y_2 + x_2^2 \cdot y_2^2 \\
&= 1 + 2x_1y_1 + 2x_2y_2 + x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 \\
&= 1 + 2\vec{x} \cdot \vec{y} + (\vec{x} \cdot \vec{y})^2 \\
&= (1 + \vec{x} \cdot \vec{y})^2
\end{aligned}$$

We realise the dot product can reduce simply to this final kernel function in the last line, which is much simpler.

Other common Kernel Functions include:

$$\text{Linear: } K(\vec{x}, \vec{y}) = \vec{x} \cdot \vec{y} \tag{2.4}$$

$$\text{Polynomial: } K(\vec{x}, \vec{y}) = (\vec{x} \cdot \vec{y} + c)^d \tag{2.5}$$

$$\text{Radial Basis Function (RBF): } K(\vec{x}, \vec{y}) = \exp(-\gamma \|\vec{x} - \vec{y}\|^2) \tag{2.6}$$

$$\text{Sigmoid: } K(\vec{x}, \vec{y}) = \tanh(\gamma \vec{x} \cdot \vec{y} + c) \tag{2.7}$$

Each kernel corresponds to a different implicit feature space. The polynomial kernel maps up to degree d, the RBF kernel maps to an infinite dimensional space but thanks to the kernel trick, we do not have to compute in such spaces.